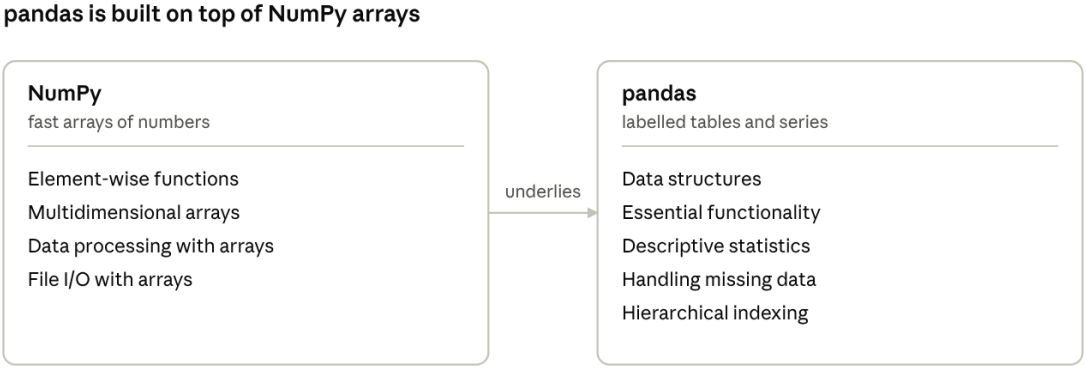
Data Science Unit 3: NumPy and pandas

Unit overview

Part	Syllabus topics	What it gives you
NumPy	Fast element-wise array functions, multidimensional arrays, data processing using arrays, file I/O with arrays	Quick maths on whole arrays without writing loops
pandas	Data structures, essential functionality, descriptive statistics, handling missing data, hierarchical indexing	Tables with row and column labels, ready for cleaning and analysis



unit roadmap · NumPy and pandas

1. NumPy Basics

NumPy's core object is the `ndarray`, a grid of values of one type stored in one block of memory, which lets it apply an operation to millions of numbers in compiled C code instead of a slow Python loop.

1.1 Why NumPy is fast

- **Homogeneous:** every element has the same type, so no per-item type checks are needed.
- **Contiguous memory:** the values sit side by side, which suits the processor cache.
- **Vectorized:** one expression such as `a * 2` runs a C loop over every element.
- **Compact:** an array of 1 million `float64` numbers uses 8 MB; the same Python list uses several times more.

```
import numpy as np

data = [1, 2, 3, 4]
arr = np.array(data)
```

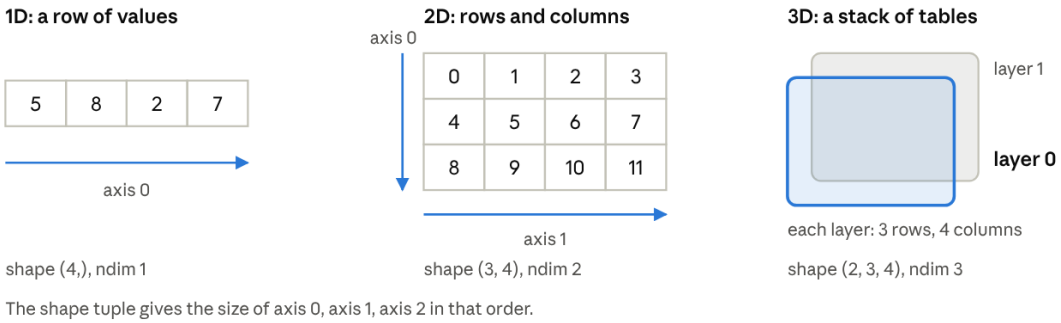
```
print(arr * 2)     # [2 4 6 8]            -> element-wise arithmetic
print(data * 2)    # [1, 2, 3, 4, 1, 2, 3, 4]  -> a list just repeats itself
print(arr.sum(), arr.mean())   # 10 2.5
```

1.2 Creating arrays and their attributes

Function	Result	Example
`np.array(seq)`	Array from a list or tuple	`np.array([1, 2, 3])`
`np.zeros(shape)` / `np.ones(shape)`	Arrays filled with 0 / 1	`np.zeros((2, 3))`
`np.arange(start, stop, step)`	Like `range`, returns an array	`np.arange(0, 10, 2)` gives `[0 2 4 6 8]`
`np.linspace(a, b, n)`	`n` evenly spaced values from `a` to `b`	`np.linspace(0, 1, 5)`
`np.eye(n)`	Identity matrix	`np.eye(3)`
`np.random.rand(...)`	Random numbers between 0 and 1	`np.random.rand(2, 2)`
`np.full(shape, v)`	Array filled with `v`	`np.full((2, 2), 7)`

Attribute	Meaning
`a.shape`	Size along each axis, as a tuple such as `(3, 4)`
`a.ndim`	Number of axes (dimensions)
`a.size`	Total number of elements
`a.dtype`	Data type of the elements
`a.itemsize`	Bytes used by one element

Arrays grow by adding axes; the shape lists the size of each one



The shape tuple gives the size of axis 0, axis 1, axis 2 in that order.

1D, 2D and 3D arrays · axes and shape

Common data types are `int32`, `int64`, `float32`, `float64`, `bool` and `str_`. Convert with `a.astype(np.float64)`; converting a float array to int drops the decimals.

1.3 Fast element-wise array functions

Operators and **universal functions (ufuncs)** act on every element at once and return a new array of the same shape.

```
a = np.array([1.0, 4.0, 9.0])
b = np.array([2.0, 2.0, 2.0])

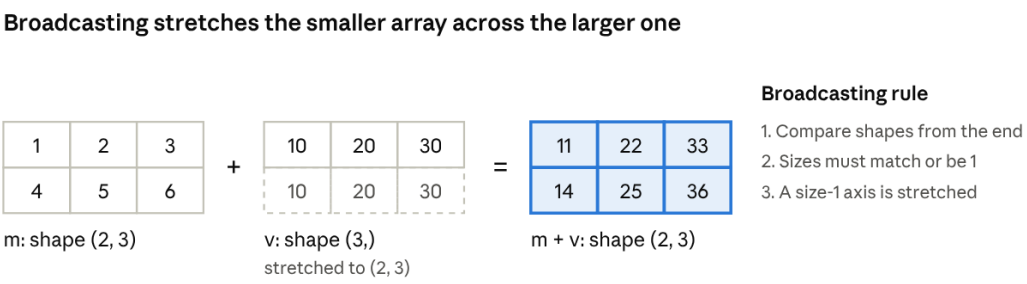
print(a + b)           # [ 3.  6. 11.]
print(a * b)           # [ 2.  8. 18.]
print(a ** 2)          # [ 1. 16. 81.]
print(np.sqrt(a))      # [1. 2. 3.]
print(np.maximum(a, 5))   # [5. 5. 9.]   -> element-wise maximum
print(a > 3)           # [False  True  True]   -> comparison gives a bool
array
```

Kind	Examples
Unary (one array)	`np.abs`, `np.sqrt`, `np.square`, `np.exp`, `np.log`, `np.sin`, `np.ceil`, `np.floor`, `np.rint`, `np.isnan`
Binary (two arrays)	`np.add`, `np.subtract`, `np.multiply`, `np.divide`, `np.power`, `np.maximum`, `np.minimum`, `np.mod`, `np.greater`, `np.logical_and`

Broadcasting lets arrays of different shapes work together. NumPy compares the shapes from the last axis backwards; two sizes are compatible when they are equal or one of them is 1, and the size-1 axis is stretched to match.

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])     # shape (2, 3)
v = np.array([10, 20, 30])    # shape (3,)

print(m + v)    # v is stretched down both rows
# [[11 22 33]
#  [14 25 36]]
print(m * 10)   # a single number is stretched everywhere
```



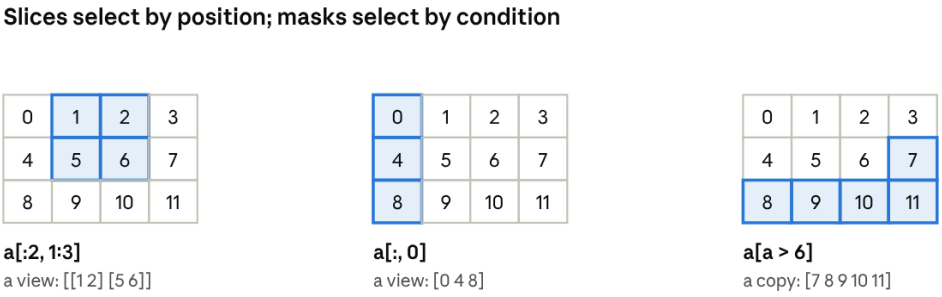
broadcasting a (3,) array over (2, 3)

1.4 Multidimensional arrays

A 2D array has rows (axis 0) and columns (axis 1); a 3D array adds a third axis (axis 0 then selects a whole 2D layer). Index with commas inside one pair of brackets: `a[row, column]`.

```
a = np.arange(12).reshape(3, 4)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

print(a[1, 2])        # 6            one element
print(a[:, 0])        # [0 4 8]      first column
print(a[:2, 1:3])     # [[1 2] [5 6]]   rows 0-1, columns 1-2
print(a[a > 6])       # [ 7  8  9 10 11]   boolean indexing
print(a[[0, 2]])      # rows 0 and 2   fancy indexing with a list
print(a.T.shape)      # (4, 3)      transpose swaps the axes
print(a.reshape(2, 6).shape)   # (2, 6)
print(a.ravel())      # flattened to 1D
```



slicing and boolean indexing · 3 examples

Indexing style	Syntax	Returns
Basic slicing	`a[1:3, :2]`	A **view**: shares memory with a
Boolean mask	`a[a > 6]`	A **copy** of the selected items
Fancy indexing	`a[[0, 2]]`	A **copy** of the selected rows

Because a slice is a view, changing it changes the original array too: `b = a[0]; b[0] = 99` also changes `a[0, 0]`. Use `a[0].copy()` when you need an independent array. To join arrays use `np.concatenate`, `np.vstack` (stack rows) and `np.hstack` (stack columns).

2. Data Processing Using Arrays and File I/O

Array-oriented programming means expressing a calculation on whole arrays instead of looping over items; it is shorter, faster and the style every later library expects.

2.1 Conditional logic with `np.where`

`np.where(condition, a, b)` is a vectorized if-else: it picks from `a` where the condition is true and from `b` elsewhere.

```
import numpy as np

temps = np.array([28, 35, 31, 22])
print(np.where(temps > 30, "hot", "ok"))      # ['ok' 'hot' 'hot' 'ok']

scores = np.array([45, 82, 67, 30])
print(np.where(scores >= 40, scores, 0))      # [45 82 67  0]
```

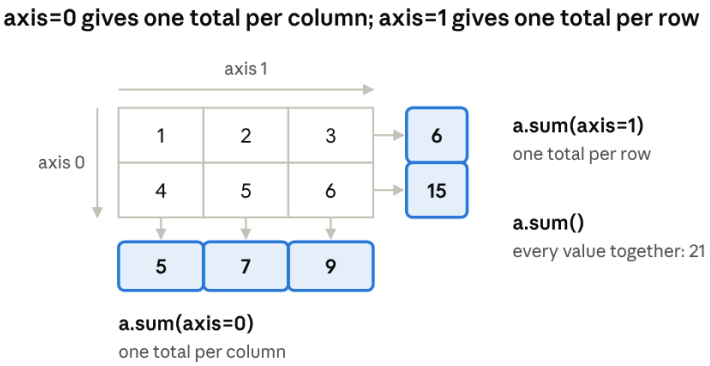
2.2 Mathematical and statistical methods

These methods summarise an array. Without an argument they use every element; the `axis` argument chooses the direction.

Method	Meaning
`sum`, `mean`, `std`, `var`	Total, average, standard deviation, variance
`min`, `max`	Smallest and largest value
`argmin`, `argmax`	Position of the smallest and largest value
`cumsum`, `cumprod`	Running total and running product

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a.sum())          # 21
print(a.sum(axis=0))    # [5 7 9]       collapse the rows: one total per
column
print(a.sum(axis=1))    # [ 6 15]       collapse the columns: one total per
row
print(a.mean(axis=0))   # [2.5 3.5 4.5]
print(a.cumsum())       # [ 1  3  6 10 15 21]
print(a.argmax())       # 5   (position in the flattened array)
```



axis=0 and axis=1 sums

2.3 Methods for boolean arrays

A comparison gives an array of `True` and `False`, which counts as 1 and 0 in arithmetic, so you can count matches with `sum()`.

```
marks = np.array([72, 35, 88, 41])
passed = marks >= 40
print(passed.sum())     # 3      how many passed
print(passed.any())     # True   at least one passed
print(passed.all())     # False  not everyone passed
```

2.4 Sorting and set logic

```
v = np.array([3, 1, 2, 3, 1])

print(np.sort(v))                    # [1 1 2 3 3]
print(np.argsort(v, kind="stable"))  # [1 4 2 0 3]   positions that would
sort v
print(np.unique(v))                  # [1 2 3]       sorted unique values
print(np.isin(v, [1, 2]))            # [False  True  True False  True]
print(np.intersect1d(v, [2, 3, 9]))  # [2 3]
```

Other set functions are `np.union1d` and `np.setdiff1d`. Use `a.sort()` to sort in place and `np.sort(a)` to get a sorted copy.

2.5 Linear algebra and random numbers

```
A = np.array([[2, 1],
              [1, 3]])
b = np.array([3, 5])

print(A @ A)                 # matrix product: [[ 5  5] [ 5 10]]
print(A.T)                   # transpose
print(np.linalg.solve(A, b)) # solves 2x + y = 3 and x + 3y = 5 -> [0.8 1.4]
print(np.linalg.det(A))      # determinant, about 5.0

rng = np.random.default_rng(seed=42)       # reproducible random numbers
print(rng.normal(size=3))                  # 3 draws from a standard normal
distribution
```

`A * A` multiplies element by element, while `A @ A` (or `np.dot`) is true matrix multiplication; do not confuse them.

2.6 File input and output with arrays

Function	Format	Use
`np.save(file, arr)` / `np.load(file)`	Binary `.npy`	Fast, exact storage of one array
`np.savez(file, a=..., b=...)`	Binary `.npz` archive	Several arrays in one file
`np.savetxt(file, arr, delimiter=",")`	Text (CSV-like)	Human-readable files
`np.loadtxt(file, delimiter=",")`	Text	Read clean numeric text files
`np.genfromtxt(file, delimiter=",")`	Text	Read text files that contain missing values

```
arr = np.arange(6).reshape(2, 3)

np.save("data.npy", arr)                # binary, keeps dtype and shape
print(np.load("data.npy"))

np.savez("pack.npz", first=arr, second=arr * 2)
pack = np.load("pack.npz")
print(pack["second"])                   # [[ 0  2  4] [ 6  8 10]]

np.savetxt("data.csv", arr, delimiter=",", fmt="%d")
print(np.loadtxt("data.csv", delimiter=","))   # values come back as floats
by default
```

3. pandas Data Structures

pandas adds labels to NumPy-style data: a `Series` is a labelled column of values, and a `DataFrame` is a table of such columns that share one row index.

3.1 Series

A Series is a one-dimensional array with an **index** of labels. Each value is reached by its label, not only by its position.

```
import pandas as pd

s = pd.Series([72, 85, 91], index=["Asha", "Ravi", "Meena"], name="marks")

print(s["Ravi"])        # 85            select by label
print(s.iloc[0])        # 72            select by position
print(s[s > 80])        # Ravi 85, Meena 91   boolean filter
print(s * 2)            # arithmetic keeps the labels
print(s.values, s.index)
```

```
d = {"CSE": 120, "IT": 90}
print(pd.Series(d))     # a dictionary becomes labels and values
```

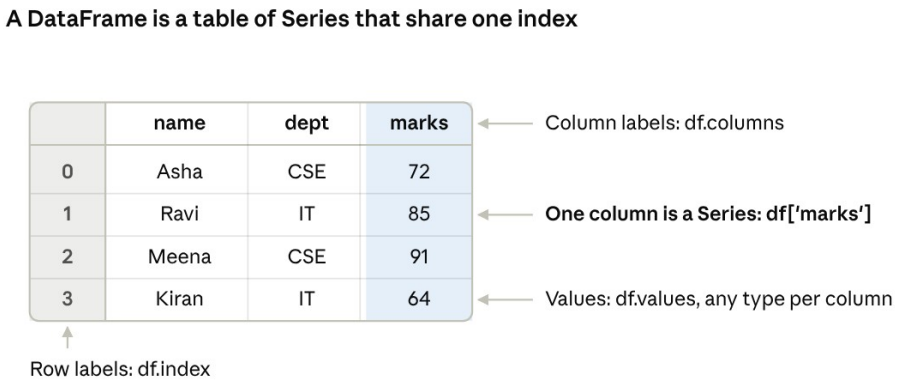
3.2 DataFrame

A DataFrame is a two-dimensional table with a row index and column labels. Each column is a Series and may have its own data type.

```
df = pd.DataFrame({
    "name":  ["Asha", "Ravi", "Meena", "Kiran"],
    "dept":  ["CSE", "IT", "CSE", "IT"],
    "marks": [72, 85, 91, 64],
})

print(df.shape)             # (4, 3)
print(df.columns.tolist())  # ['name', 'dept', 'marks']
print(df.dtypes)            # name object, dept object, marks int64
print(df.head(2))           # first 2 rows
df.info()                   # columns, types, non-null counts

df["passed"] = df["marks"] >= 40    # add a column
del df["passed"]                    # remove it again
```



DataFrame anatomy · index, columns, values

Common ways to build a DataFrame: a dictionary of lists (above), a list of dictionaries (one per row), a NumPy array with `columns=[...]`, or a file with `pd.read_csv("file.csv")`.

3.3 Index objects

The row labels and column labels are `Index` objects. They are immutable (you replace them instead of editing one label) and they may contain duplicates, although unique labels are easier to work with.

```
df2 = df.set_index("name")        # use a column as the row labels
print(df2.loc["Asha", "marks"])    # 72
print(df2.reset_index().columns.tolist())   # ['name', 'dept', 'marks'] ->
back to a default 0..n-1 index
```

3.4 Selecting data

Goal	Code	Returns
One column	`df["marks"]`	Series
Several columns	`df[["name", "marks"]]`	DataFrame
Rows by label	`df.loc[0:2]`	DataFrame; the end label **is included**
Rows and columns by label	`df.loc[1, "marks"]`	One value
Rows by position	`df.iloc[0:2]`	DataFrame; the end position is **excluded**
Rows and columns by position	`df.iloc[1, 2]`	One value
Rows by condition	`df[df["marks"] > 70]`	Rows where the test is true
Combined conditions	`df[(df.marks > 60) & (df.dept == "IT")]`	Use `&`, `|` and `~` with brackets, not `and` or `or`

4. Essential Functionality and Descriptive Statistics

The everyday pandas toolkit is reindexing, label-based arithmetic, applying functions, sorting and summarising; most of it works the same on a Series and a DataFrame.

4.1 Reindexing and dropping

`reindex` rearranges data to a new list of labels, inserting `NaN` (not a number, meaning missing) for labels that do not exist. `drop` removes rows or columns by label and returns a new object.

```
s = pd.Series([4.5, 7.2, -5.3], index=["c", "a", "b"])

print(s.reindex(["a", "b", "c", "d"]))                # d becomes NaN
print(s.reindex(["a", "b", "c", "d"], fill_value=0))  # d becomes 0

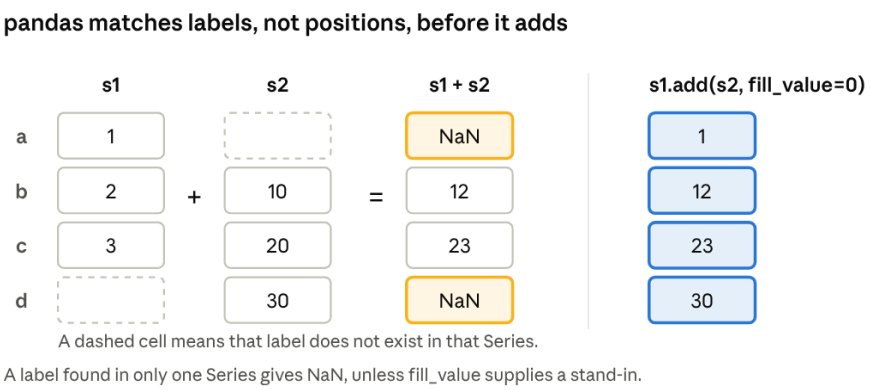
print(s.drop("a"))                                    # remove one row label
print(df.drop(columns=["dept"]))                      # remove a column
print(df.drop(index=[0, 1]))                          # remove rows 0 and 1
```

4.2 Arithmetic and data alignment

When you combine two objects, pandas first **aligns them on their labels**. A label found in only one object gives `NaN` in the result.

```
s1 = pd.Series([1, 2, 3], index=["a", "b", "c"])
s2 = pd.Series([10, 20, 30], index=["b", "c", "d"])

print(s1 + s2)                    # a NaN, b 12, c 23, d NaN
print(s1.add(s2, fill_value=0))   # a 1, b 12, c 23, d 30   treat a missing
label as 0
```



label alignment · s1 + s2

The methods `add`, `sub`, `mul` and `div` accept `fill_value`. A DataFrame combined with a Series is aligned on the columns by default and broadcast down the rows, as in NumPy.

4.3 Function application and mapping

- `df.apply(func, axis=0)` runs a function on each column (or each row with `axis=1`).
- `series.map(func_or_dict)` transforms every value of a Series, for example `df["dept"].map({"CSE": "Computer", "IT": "Info Tech"})`.
- `df.map(func)` applies a function to every single cell (older versions call this `applymap`).
- Many NumPy functions such as `np.sqrt(df["marks"])` work directly on pandas objects.

```
rng = lambda col: col.max() - col.min()      # a function of one column
print(df[["marks"]].apply(rng))               # marks    27
print(df["marks"].map(lambda m: "P" if m >= 40 else "F").tolist())   # ['P',
'P', 'P', 'P']
```

4.4 Sorting and ranking

```
print(df.sort_values("marks", ascending=False))         # highest marks first
print(df.sort_values(["dept", "marks"]))                # by department, then
marks
print(df.set_index("name").sort_index())                # sort by row labels
print(df["marks"].rank(ascending=False))                # 3.0, 2.0, 1.0, 4.0
-> rank 1 is the highest
```

`rank` gives each value its position in the sorted order; tied values share the average rank by default.

4.5 Summarizing and computing descriptive statistics

pandas statistics skip missing values by default (`skipna=True`). On a DataFrame they work column by column; use `axis=1` to work across each row.

Method	What it returns
`count()`	Number of non-missing values
`sum()`, `mean()`, `median()`	Total, average, middle value

Method	What it returns
`min()`, `max()`, `idxmin()`, `idxmax()`	Extremes, and the **label** where they occur
`std()`, `var()`	Sample standard deviation and variance
`quantile(q)`	Value below which a fraction `q` of the data lies
`cumsum()`, `cumprod()`	Running totals and products
`describe()`	Count, mean, std, min, quartiles and max in one table
`corr()`, `cov()`	Correlation and covariance between columns
`value_counts()`, `unique()`, `nunique()`	Frequency of each value, distinct values, number of distinct values

```
print(df["marks"].describe())
# count     4.000000
# mean     78.000000
# std      12.247449
# min      64.000000
# 25%      70.000000
# 50%      78.500000
# 75%      86.500000
# max      91.000000

print(df["marks"].idxmax())          # 2   -> row label of the highest mark
print(df["dept"].value_counts())     # CSE 2, IT 2
print(df[["marks"]].corr())          # correlation matrix of numeric columns
```

5. Handling Missing Data

Real data always has gaps; pandas marks them as missing values so you can find them, drop them or fill them in, and so statistics do not break.

5.1 How missing data is represented

- **`NaN`** (not a number) marks a missing number in a float column.
- **`None`** marks a missing item in a text (object) column, and pandas treats it as missing too.
- **`NaT`** marks a missing date or time.
- **`pd.NA`** is the missing marker for the newer nullable data types.

`isnull()` (alias `isna()`) returns `True` where a value is missing, and `notnull()` is its opposite. Because `True` counts as 1, `df.isnull().sum()` gives the number of gaps in each column.

```
import numpy as np
import pandas as pd

data = pd.DataFrame({
    "name":  ["Asha", "Ravi", "Meena", "Kiran"],
    "marks": [72, np.nan, 91, np.nan],
    "city":  ["Kochi", None, "Pune", "Delhi"],
```

```
})

print(data.isnull().sum())      # name 0, marks 2, city 1
print(data["marks"].mean())     # 81.5   missing values are skipped
```

5.2 Filtering out missing data

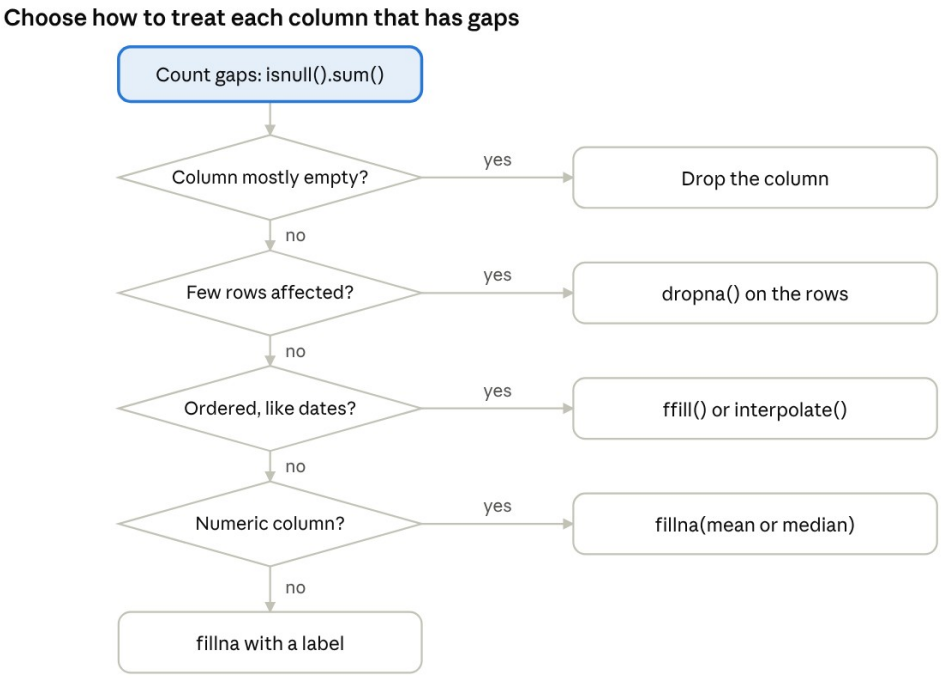
```
print(data.dropna())                        # keeps only complete rows (Asha
and Meena)
print(data.dropna(how="all"))               # drops a row only if every value
is missing
print(data.dropna(subset=["marks"]))        # looks only at the marks column
print(data.dropna(axis=1))                  # drops columns that contain any
gap -> only name remains
print(data.dropna(thresh=3))                # keep rows that have at least 3
real values
```

5.3 Filling in missing data

```
print(data.fillna(0))                                    # one value for
every gap
print(data.fillna({"marks": data["marks"].mean(),        # a different value
per column
                   "city": "Unknown"}))

vals = pd.Series([1, np.nan, np.nan, 4])
print(vals.ffill().tolist())            # [1.0, 1.0, 1.0, 4.0]   copy the
last valid value forward
print(vals.bfill().tolist())            # [1.0, 4.0, 4.0, 4.0]   copy the
next valid value backward
print(vals.interpolate().tolist())      # [1.0, 2.0, 3.0, 4.0]   straight-
line estimate
```

Situation	Usual choice
Few rows affected and the data is plentiful	`dropna()`
A whole column is mostly empty	Drop the column
A numeric column with random gaps	Fill with the mean or median
A category column	Fill with a label such as `"Unknown"` or the most frequent value
An ordered series such as daily readings	`ffill()` or `interpolate()`



missing-data decision path · 4 questions

Always record what you did, because filling values changes the statistics of the data.

6. Hierarchical Indexing

Hierarchical indexing (a `MultiIndex`) gives an object two or more levels of row labels, so a table with several keys, such as department and year, can be handled as one structure.

6.1 MultiIndex on a Series

```
s = pd.Series([10, 20, 30, 40],
              index=[["CSE", "CSE", "IT", "IT"],
                     [2025, 2026, 2025, 2026]])

print(s["CSE"])          # 2025 10, 2026 20      select the outer level
print(s.loc[:, 2026])    # CSE 20, IT 40        select the inner level
print(s.unstack())       # the inner level becomes the columns
#      2025  2026
# CSE    10    20
# IT     30    40
```

↗ stack and unstack · MultiIndex

`unstack()` pivots an index level into columns, and `stack()` does the reverse.

6.2 MultiIndex on a DataFrame

```
df = pd.DataFrame({
    "dept": ["CSE", "CSE", "IT", "IT"],
```

```
    "year": [2025, 2026, 2025, 2026],
    "students": [10, 20, 30, 40],
}).set_index(["dept", "year"])

print(df.loc["CSE"])                       # all years for CSE
print(df.loc[("IT", 2026), "students"])    # 40   a tuple selects one exact row
print(df.groupby(level="dept").sum())      # CSE 30, IT 70   statistics by one level
print(df.swaplevel().sort_index())         # year becomes the outer level
print(df.reset_index())                    # turn the levels back into ordinary columns
```

Operation	Purpose
`set_index(["a", "b"])`	Make columns into index levels
`reset_index()`	Move index levels back into columns
`unstack()` / `stack()`	Move a level between the rows and the columns
`swaplevel()` and `sort_index()`	Reorder the levels and sort so slicing works well
`loc[(outer, inner)]`	Select with a tuple of labels
`groupby(level=...)`	Summarise by one level

Columns can also have two levels, which is how a pivot table with several value columns is displayed.

Quick revision

Remember one core idea for each topic.

- **ndarray:** a fixed-type grid in contiguous memory; check `shape`, `ndim` and `dtype` first.
- **Element-wise functions:** operators and ufuncs such as `np.sqrt` and `np.maximum` act on every element without a loop.
- **Broadcasting:** compare shapes from the last axis; sizes must be equal or 1, and a size-1 axis is stretched.
- **Indexing:** slices are views (shared memory); boolean masks and fancy indexing return copies.
- **Axis:** `axis=0` runs down the rows and gives one result per column; `axis=1` runs across the columns and gives one result per row.
- **Data processing:** `np.where` for if-else, `sum`, `mean`, `std`, `argmax`, `sort`, `unique` and `@` for matrix products.
- **File I/O:** `np.save` and `np.load` for binary `.npy`, `np.savez` for several arrays, `np.savetxt`, `np.loadtxt` and `np.genfromtxt` for text.
- **Series and DataFrame:** a labelled 1D array and a 2D table whose columns are Series.
- **Selecting:** `loc` uses labels (end included); `iloc` uses positions (end excluded).
- **Alignment:** arithmetic matches labels first, and unmatched labels give `NaN`; `fill_value` avoids that.
- **Descriptive statistics:** `describe`, `mean`, `std`, `corr`, `value_counts`; missing values are skipped by default.

- **Missing data:** `isnull`, `dropna`, `fillna`, `ffill`, `interpolate`.
- **Hierarchical indexing:** a MultiIndex gives several label levels; `unstack` and `stack` move a level between rows and columns.

Likely exam questions

Short answer (2 to 3 marks)

1. What is an ndarray? Name any four of its attributes.
2. What is broadcasting? Give an example.
3. Differentiate between a view and a copy in NumPy.
4. What does `np.where` do?
5. Differentiate between `loc` and `iloc`.
6. What is a Series? How does it differ from a NumPy array?
7. Write the code to count the missing values in each column of a DataFrame.

Long answer (5 to 10 marks)

1. Explain the creation of NumPy arrays and the fast element-wise functions, with examples.
2. Explain multidimensional array indexing and slicing, with examples.
3. Describe the mathematical and statistical methods, sorting and file I/O available for NumPy arrays.
4. Explain the pandas data structures Series and DataFrame, with examples.
5. Explain reindexing, data alignment and sorting in pandas.
6. Explain how pandas computes descriptive statistics, with an example using `describe`.
7. Explain the methods for handling missing data.
8. Explain hierarchical indexing and the use of `stack` and `unstack`.

References

Text books

1. Mark Lutz, *Learning Python*, O'Reilly.
2. Mark Lutz, *Programming Python*, O'Reilly.
3. Wes McKinney, *Python for Data Analysis*, O'Reilly (NumPy basics, pandas data structures, descriptive statistics, missing data and hierarchical indexing).

Reference books

1. Martin C. Brown, *Python: The Complete Reference*, McGraw Hill Education.
2. Paul Barry, *Head First Python*, O'Reilly.